

周计划算法 2026-06-29 ~ 2026-07-03

自动生成于 2026-06-28 | 速刷 Hot 100

2026-06-29 周一

新题 (学习)

LC 0114 二叉树展开为链表

题目描述： 二叉树展开为链表

示例： `root=[1,2,5,3,4,null,6] => [1,null,2,null,3,null,4,null,5,null,6]`

LeetCode: <https://leetcode.cn/problems/flatten-binary-tree-to-linked-list/>

算法核心： 前序 DFS：先递归展开左右子树，再将 `root.right` 接到 `root.left` 最右节点，最后 `root.right = root.left; root.left = null`

复杂度： 时间 $O(n)$ ，空间 $O(n)$

标签： 二叉树，DFS，链表

代码实现：

```
public void flatten(TreeNode root) {
    if (root == null) return;
    flatten(root.left);
    flatten(root.right);

    TreeNode left = root.left;
    TreeNode right = root.right;
    root.left = null;
    root.right = left;

    TreeNode curr = root;
    while (curr.right != null) curr = curr.right;
    curr.right = right;
}
```

LC 0098 验证二叉搜索树

题目描述： 验证二叉搜索树

示例： `root=[2,1,3] => true; root=[5,1,4,null,null,3,6] => false`

LeetCode: <https://leetcode.cn/problems/validate-binary-search-tree/>

算法核心： 递归传上下界 (min, max)，节点值必须在开区间内；或中序遍历结果严格递增

复杂度： 时间 $O(n)$, 空间 $O(n)$

标签： 二叉树, DFS, BST

代码实现：

```
public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

private boolean validate(TreeNode node, Integer min, Integer max) {
    if (node == null) return true;
    if ((min != null && node.val <= min) || (max != null && node.val >= max)) {
        return false;
    }
    return validate(node.left, min, node.val)
        && validate(node.right, node.val, max);
}
```

 **复习**

LC 0021 合并两个有序链表 (R4)

题目描述： 合并两个有序链表

示例： list1=[1,2,4], list2=[1,3,4] => [1,1,2,3,4,4]

LeetCode: <https://leetcode.cn/problems/>

算法核心： dummy 头节点 + 双指针遍历两链表，取较小值接入结果链

复杂度： 时间 $O(n+m)$, 空间 $O(1)$

标签： 链表, 递归

代码实现：

```
public class LC_0021_MergeTwoSortedLists {

    public ListNode mergeTwoLists(ListNode list1, ListNode list2) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;

        while (list1 != null && list2 != null) {
            if (list1.val <= list2.val) {
                curr.next = list1;
                list1 = list1.next;
            } else {
                curr.next = list2;
                list2 = list2.next;
            }
            curr = curr.next;
        }
        curr.next = (list1 != null) ? list1 : list2;
        return dummy.next;
    }
}
```

```

public static void main(String[] args) {
    LC_0021_MergeTwoSortedLists s = new LC_0021_MergeTwoSortedLists();
    System.out.println(java.util.Arrays.toString(ListNode.toArray(
        s.mergeTwoLists(
            ListNode.fromArray(new int[]{1, 2, 4}),
            ListNode.fromArray(new int[]{1, 3, 4})))));
}
}

```

LC 0023 合并 K 个升序链表 (R4)

题目描述： 合并 K 个升序链表

示例： `lists=[[1,4,5],[1,3,4],[2,6]] => [1,1,2,3,4,4,5,6]`

LeetCode： <https://leetcode.cn/problems/>

算法核心： 小顶堆：K 个头节点入堆，每次弹出最小节点接入结果链，并将其 next 入堆

复杂度： 时间 $O(n \log K)$ ，空间 $O(K)$

标签： 链表，分治，堆

代码实现：

```

public class LC_0023_MergeKSortedLists {

    public ListNode mergeKLists(ListNode[] lists) {
        if (lists == null || lists.length == 0) {
            return null;
        }
        PriorityQueue<ListNode> pq = new PriorityQueue<>((a, b) -> a.val - b.val);
        for (ListNode node : lists) {
            if (node != null) {
                pq.offer(node);
            }
        }

        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
        while (!pq.isEmpty()) {
            ListNode node = pq.poll();
            curr.next = node;
            curr = curr.next;
            if (node.next != null) {
                pq.offer(node.next);
            }
        }
        return dummy.next;
    }

    public static void main(String[] args) {
        LC_0023_MergeKSortedLists s = new LC_0023_MergeKSortedLists();
        ListNode[] lists = {
            ListNode.fromArray(new int[]{1, 4, 5}),
            ListNode.fromArray(new int[]{1, 3, 4}),
            ListNode.fromArray(new int[]{2, 6})
        };
        System.out.println(java.util.Arrays.toString(ListNode.toArray(s.mergeKLists(lists))))
    }
}

```

```
}  
}
```

2026-06-30 周二

📌 新题（学习）

LC 0230 二叉搜索树中第 K 小的元素

题目描述： 二叉搜索树中第 K 小的元素

示例： `root=[3,1,4,null,2]`, `k=1 => 1`

LeetCode: <https://leetcode.cn/problems/kth-smallest-element-in-a-bst/>

算法核心： 中序遍历 BST 得到升序序列，数到第 k 个即答案

复杂度： 时间 $O(n)$ ，空间 $O(n)$

标签： 二叉树，BST，中序遍历

代码实现：

```
public int kthSmallest(TreeNode root, int k) {  
    Deque stack = new ArrayDeque<>();  
    TreeNode curr = root;  
    while (curr != null || !stack.isEmpty()) {  
        while (curr != null) {  
            stack.push(curr);  
            curr = curr.left;  
        }  
        curr = stack.pop();  
        if (--k == 0) return curr.val;  
        curr = curr.right;  
    }  
    return -1;  
}
```

LC 0199 二叉树的右视图

题目描述： 二叉树的右视图

示例： `root=[1,2,3,null,5,null,4]` => `[1,3,4]`

LeetCode: <https://leetcode.cn/problems/binary-tree-right-side-view/>

算法核心： BFS 层序遍历，每层最后一个节点；或 DFS 先访问右子树，按深度首次记录

复杂度： 时间 $O(n)$ ，空间 $O(n)$

标签： 二叉树，BFS，DFS

代码实现：

```

public List<Integer> rightSideView(TreeNode root) {
    List<Integer> res = new ArrayList<>();
    if (root == null) return res;

    Queue<TreeNode> q = new LinkedList<>();
    q.offer(root);
    while (!q.isEmpty()) {
        int size = q.size();
        for (int i = 0; i < size; i++) {
            TreeNode node = q.poll();
            if (node.left != null) q.offer(node.left);
            if (node.right != null) q.offer(node.right);
            if (i == size - 1) res.add(node.val);
        }
    }
    return res;
}

```

复习

LC 0025 K 个一组翻转链表 (R4)

题目描述： K 个一组翻转链表

示例： head=[1,2,3,4,5], k=2 => [2,1,4,3,5]

LeetCode： <https://leetcode.cn/problems/>

算法核心： dummy 头节点，每次定位 K 个节点区间并翻转，不足 K 不翻转

复杂度： 时间 O(n)，空间 O(1)

标签： 链表，翻转

代码实现：

```

public class LC_0025_ReverseNodesInKGroup {

    public ListNode reverseKGroup(ListNode head, int k) {
        ListNode dummy = new ListNode(0, head);
        ListNode prev = dummy;

        while (true) {
            ListNode end = prev;
            for (int i = 0; i < k && end != null; i++) {
                end = end.next;
            }
            if (end == null) {
                break;
            }

            ListNode start = prev.next;
            ListNode next = end.next;

            ListNode curr = start;
            ListNode prevNode = next;
            while (curr != next) {

```

```

        ListNode tmp = curr.next;
        curr.next = prevNode;
        prevNode = curr;
        curr = tmp;
    }

    prev.next = end;
    prev = start;
}
return dummy.next;
}

public static void main(String[] args) {
    LC_0025_ReverseNodesInKGroup s = new LC_0025_ReverseNodesInKGroup();
    System.out.println(java.util.Arrays.toString(ListNode.toArray(
        s.reverseKGroup(ListNode.fromArray(new int[]{1, 2, 3, 4, 5}), 2))));
}
}

```

LC 0148 排序链表 (R4)

题目描述： 排序链表

示例： head=[4,2,1,3] => [1,2,3,4]

LeetCode： <https://leetcode.cn/problems/>

算法核心： 归并排序：快慢指针找中点切分，递归排序后合并

复杂度： 时间 $O(n \log n)$ ，空间 $O(\log n)$

标签： 链表，排序，归并

代码实现：

```

public class LC_0148_SortList {

    public ListNode sortList(ListNode head) {
        if (head == null || head.next == null) {
            return head;
        }

        ListNode slow = head;
        ListNode fast = head.next;
        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }
        ListNode mid = slow.next;
        slow.next = null;

        ListNode left = sortList(head);
        ListNode right = sortList(mid);
        return merge(left, right);
    }

    private ListNode merge(ListNode l1, ListNode l2) {
        ListNode dummy = new ListNode(0);
        ListNode curr = dummy;
    }
}

```

```

        while (l1 != null && l2 != null) {
            if (l1.val <= l2.val) {
                curr.next = l1;
                l1 = l1.next;
            } else {
                curr.next = l2;
                l2 = l2.next;
            }
            curr = curr.next;
        }
        curr.next = (l1 != null) ? l1 : l2;
        return dummy.next;
    }

    public static void main(String[] args) {
        LC_0148_SortList s = new LC_0148_SortList();
        System.out.println(java.util.Arrays.toString(ListNode.toArray(
            s.sortList(ListNode.fromArray(new int[]{4, 2, 1, 3})))));
    }
}

```

2026-07-01 周三

新题 (学习)

LC 0112 路径总和

题目描述：路径总和

示例： root=[5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum=22 => true

LeetCode: <https://leetcode.cn/problems/path-sum/>

算法核心：DFS 递归，targetSum -= node.val，到叶子且剩余为 0 则命中

复杂度：时间 O(n)，空间 O(n)

标签：  二叉树,  DFS

代码实现：

```

public boolean hasPathSum(TreeNode root, int targetSum) {
    if (root == null) return false;
    if (root.left == null && root.right == null) {
        return targetSum == root.val;
    }
    int remain = targetSum - root.val;
    return hasPathSum(root.left, remain) || hasPathSum(root.right, remain);
}

```

LC 0437 路径总和 III

题目描述：路径总和 III

示例: `root=[10,5,-3,3,2,null,11,3,-2,null,1], targetSum=8 => 3`

LeetCode: <https://leetcode.cn/problems/path-sum-iii/>

算法核心: 双重 DFS + 前缀和 HashMap: 以每个节点为起点向下枚举, 用 prefixSum 计数

复杂度: 时间 $O(n)$, 空间 $O(n)$

标签: 二叉树, DFS, 前缀和, 哈希表

代码实现:

```
public int pathSum(TreeNode root, int targetSum) {
    if (root == null) return 0;
    Map<Long, Integer> prefix = new HashMap<>();
    prefix.put(0L, 1);
    return dfs(root, 0L, targetSum, prefix)
        + pathSum(root.left, targetSum)
        + pathSum(root.right, targetSum);
}

private int dfs(TreeNode node, long curr, int target,
    Map<Long, Integer> prefix) {
    if (node == null) return 0;
    curr += node.val;
    int count = prefix.getOrDefault(curr - target, 0);
    prefix.merge(curr, 1, Integer::sum);
    count += dfs(node.left, curr, target, prefix);
    count += dfs(node.right, curr, target, prefix);
    prefix.merge(curr, -1, Integer::sum);
    return count;
}
```

 复习

LC 0094 二叉树的中序遍历 (R4)

题目描述: 二叉树的中序遍历

示例: `root=[1,null,2,3] => [1,3,2]`

LeetCode: <https://leetcode.cn/problems/>

算法核心: 迭代栈模拟: 先深入左子树, 出栈访问, 再进入右子树

复杂度: 时间 $O(n)$, 空间 $O(n)$

标签: 二叉树, 遍历, 栈

代码实现:

```
public class LC_0094_BinaryTreeInorderTraversal {

    public List<Integer> inorderTraversal(TreeNode root) {
        List<Integer> res = new ArrayList<>();
        Deque<TreeNode> stack = new ArrayDeque<>();
        TreeNode curr = root;
        while (curr != null || !stack.isEmpty()) {

```



```

        while (curr != null) {
            stack.push(curr);
            curr = curr.left;
        }
        curr = stack.pop();
        res.add(curr.val);
        curr = curr.right;
    }
    return res;
}

public static void main(String[] args) {
    LC_0094_BinaryTreeInorderTraversal s = new LC_0094_BinaryTreeInorderTraversal();
    TreeNode root = new TreeNode(1, null, new TreeNode(2, new TreeNode(3), null));
    System.out.println(s.inorderTraversal(root));
}
}

```

LC 0104 二叉树的最大深度 (R4)

题目描述： 二叉树的最大深度

示例： root=[3,9,20,null,null,15,7] => 3

LeetCode： <https://leetcode.cn/problems/>

算法核心： BFS 层序遍历统计层数

复杂度： 时间 $O(n)$, 空间 $O(n)$

标签： 二叉树, DFS, BFS

代码实现：

```

public class LC_0104_MaximumDepthOfBinaryTree {

    public int maxDepth(TreeNode root) {
        if (root == null) {
            return 0;
        }
        Queue q = new LinkedList<>();
        q.offer(root);
        int depth = 0;
        while (!q.isEmpty()) {
            int size = q.size();
            depth++;
            for (int i = 0; i < size; i++) {
                TreeNode node = q.poll();
                if (node.left != null) {
                    q.offer(node.left);
                }
                if (node.right != null) {
                    q.offer(node.right);
                }
            }
        }
        return depth;
    }
}

```

```
public static void main(String[] args) {
    LC_0104_MaximumDepthOfBinaryTree s = new LC_0104_MaximumDepthOfBinaryTree();
    TreeNode root = TreeNode.fromArray(new Integer[]{3, 9, 20, null, null, 15, 7});
    System.out.println(s.maxDepth(root));
}
}
```

2026-07-02 周四

新题 (学习)

LC 0236 二叉树的最近公共祖先

题目描述： 二叉树的最近公共祖先

示例： root=[3,5,1,6,2,0,8,null,null,7,4], p=5, q=1 => 3

LeetCode: <https://leetcode.cn/problems/lowest-common-ancestor-of-a-binary-tree/>

算法核心： 后序 DFS： 左右子树各返回找到的节点；若当前节点是 p/q 或左右各找到一个，则当前为 LCA

复杂度： 时间 $O(n)$ ，空间 $O(n)$

标签： 二叉树，DFS

代码实现：

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

LC 0124 二叉树中的最大路径和

题目描述： 二叉树中的最大路径和

示例： root=[-10,9,20,null,null,15,7] => 42

LeetCode: <https://leetcode.cn/problems/binary-tree-maximum-path-sum/>

算法核心： 后序 DFS： 返回以当前节点为端点的最大单边贡献，用 left+val+right 更新全局最大值

复杂度： 时间 $O(n)$ ，空间 $O(n)$

标签： 二叉树，DFS，分治

代码实现：

```
private int maxSum = Integer.MIN_VALUE;
```

```

public int maxPathSum(TreeNode root) {
    maxGain(root);
    return maxSum;
}

private int maxGain(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(maxGain(node.left), 0);
    int right = Math.max(maxGain(node.right), 0);
    maxSum = Math.max(maxSum, node.val + left + right);
    return node.val + Math.max(left, right);
}

```

复习

LC 0101 对称二叉树 (R4)

题目描述： 对称二叉树

示例： root=[1,2,2,3,4,4,3] => true

LeetCode： <https://leetcode.cn/problems/>

算法核心： 递归判断左右子树是否镜像对称

复杂度： 时间 $O(n)$, 空间 $O(n)$

标签： 二叉树, DFS, BFS

代码实现：

```

public class LC_0101_SymmetricTree {

    public boolean isSymmetric(TreeNode root) {
        if (root == null) {
            return true;
        }
        return isMirror(root.left, root.right);
    }

    private boolean isMirror(TreeNode t1, TreeNode t2) {
        if (t1 == null && t2 == null) {
            return true;
        }
        if (t1 == null || t2 == null) {
            return false;
        }
        return t1.val == t2.val
            && isMirror(t1.left, t2.right)
            && isMirror(t1.right, t2.left);
    }

    public static void main(String[] args) {
        LC_0101_SymmetricTree s = new LC_0101_SymmetricTree();
        TreeNode root = TreeNode.fromArray(new Integer[]{1, 2, 2, 3, 4, 4, 3});
        System.out.println(s.isSymmetric(root));
    }
}

```

```
}  
}
```

LC 0226 翻转二叉树 (R4)

题目描述： 翻转二叉树

示例： `root=[4,2,7,1,3,6,9] => [4,7,2,9,6,3,1]`

LeetCode: <https://leetcode.cn/problems/>

算法核心： 递归交换左右子树

复杂度： 时间 $O(n)$, 空间 $O(n)$

标签： 二叉树, DFS

代码实现：

```
public class LC_0226_InvertBinaryTree {  
  
    public TreeNode invertTree(TreeNode root) {  
        if (root == null) {  
            return null;  
        }  
        TreeNode left = invertTree(root.right);  
        TreeNode right = invertTree(root.left);  
        root.left = left;  
        root.right = right;  
        return root;  
    }  
  
    public static void main(String[] args) {  
        LC_0226_InvertBinaryTree s = new LC_0226_InvertBinaryTree();  
        TreeNode root = TreeNode.fromArray(new Integer[]{4, 2, 7, 1, 3, 6, 9});  
        System.out.println(java.util.Arrays.toString(TreeNode.toArray(s.invertTree(root))));  
    }  
}
```

2026-07-03 周五

 新题 (学习)

LC 0020 有效的括号

题目描述： 有效的括号

示例： `s="()[]{}" => true; s="[]" => false`

LeetCode: <https://leetcode.cn/problems/valid-parentheses/>

算法核心： 栈：左括号入栈，右括号与栈顶匹配弹出；最终栈空则有效

复杂度： 时间 $O(n)$, 空间 $O(n)$

标签： 栈, 字符串

代码实现：

```
public boolean isValid(String s) {
    Deque<Character> stack = new ArrayDeque<>();
    for (char c : s.toCharArray()) {
        if (c == '(' || c == '[' || c == '{') {
            stack.push(c);
        } else {
            if (stack.isEmpty()) return false;
            char top = stack.pop();
            if ((c == ')' && top != '(')
                || (c == ']' && top != '[')
                || (c == '}' && top != '{')) {
                return false;
            }
        }
    }
    return stack.isEmpty();
}
```

LC 0155 最小栈

题目描述： 最小栈

示例： push(-2), push(0), push(-3), getMin()=>-3, pop(), top()=>0, getMin()=>-2

LeetCode： <https://leetcode.cn/problems/min-stack/>

算法核心： 辅助栈同步记录当前最小值; push 时若 $val \leq min$ 则同步入 minStack

复杂度： 时间 $O(1)$, 空间 $O(n)$

标签： 栈, 设计

代码实现：

```
class MinStack {
    private final Deque<Integer> stack = new ArrayDeque<>();
    private final Deque<Integer> minStack = new ArrayDeque<>();

    public void push(int val) {
        stack.push(val);
        if (minStack.isEmpty() || val <= minStack.peek()) {
            minStack.push(val);
        }
    }

    public void pop() {
        if (stack.pop().equals(minStack.peek())) {
            minStack.pop();
        }
    }

    public int top() { return stack.peek(); }
}
```

```
public int getMin() { return minStack.peek(); }  
}
```

复习

LC 0102 二叉树的层序遍历 (R4)

题目描述： 二叉树的层序遍历

示例： root=[3,9,20,null,null,15,7] => [[3],[9,20],[15,7]]

LeetCode: <https://leetcode.cn/problems/>

算法核心： BFS 队列： 每轮处理当前层所有节点

复杂度： 时间 $O(n)$, 空间 $O(n)$

标签： 二叉树, BFS, 队列

代码实现：

```
public class LC_0102_BinaryTreeLevelOrderTraversal {  
  
    public List<List<Integer>> levelOrder(TreeNode root) {  
        List<List<Integer>> res = new ArrayList<>();  
        if (root == null) {  
            return res;  
        }  
  
        Queue<TreeNode> q = new LinkedList<>();  
        q.offer(root);  
  
        while (!q.isEmpty()) {  
            int size = q.size();  
            List<Integer> level = new ArrayList<>();  
            for (int i = 0; i < size; i++) {  
                TreeNode node = q.poll();  
                level.add(node.val);  
                if (node.left != null) {  
                    q.offer(node.left);  
                }  
                if (node.right != null) {  
                    q.offer(node.right);  
                }  
            }  
            res.add(level);  
        }  
        return res;  
    }  
  
    public static void main(String[] args) {  
        LC_0102_BinaryTreeLevelOrderTraversal s = new LC_0102_BinaryTreeLevelOrderTraversal();  
        TreeNode root = TreeNode.fromArray(new Integer[]{3, 9, 20, null, null, 15, 7});  
        System.out.println(s.levelOrder(root));  
    }  
}
```

LC 0105 从前序与中序遍历序列构造二叉树 (R4)

题目描述： 从前序与中序遍历序列构造二叉树

示例： preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]

LeetCode： <https://leetcode.cn/problems/>

算法核心： 前序首元素为根，在中序中定位根划分左右子树，递归构建

复杂度： 时间 $O(n)$, 空间 $O(n)$

标签： 二叉树, 分治, 哈希表

代码实现：

```
public class LC_0105_ConstructBinaryTreeFromPreorderAndInorderTraversal {

    public TreeNode buildTree(int[] preorder, int[] inorder) {
        Map<Integer, Integer> idxMap = new HashMap<>();
        for (int i = 0; i < inorder.length; i++) {
            idxMap.put(inorder[i], i);
        }
        return build(preorder, 0, preorder.length - 1,
                    inorder, 0, inorder.length - 1, idxMap);
    }

    private TreeNode build(int[] pre, int pL, int pR,
                          int[] in, int iL, int iR,
                          Map<Integer, Integer> map) {
        if (pL > pR || iL > iR) {
            return null;
        }

        int rootVal = pre[pL];
        TreeNode root = new TreeNode(rootVal);
        int rootIdx = map.get(rootVal);
        int leftSize = rootIdx - iL;

        root.left = build(pre, pL + 1, pL + leftSize,
                        in, iL, rootIdx - 1, map);
        root.right = build(pre, pL + leftSize + 1, pR,
                        in, rootIdx + 1, iR, map);
        return root;
    }

    public static void main(String[] args) {
        LC_0105_ConstructBinaryTreeFromPreorderAndInorderTraversal s =
            new LC_0105_ConstructBinaryTreeFromPreorderAndInorderTraversal();
        TreeNode root = s.buildTree(
            new int[]{3, 9, 20, 15, 7},
            new int[]{9, 3, 15, 20, 7});
        System.out.println(java.util.Arrays.toString(TreeNode.toArray(root)));
    }
}
```



本周算法汇总

学习节奏： 每日 2 道新题 + 2 道复盘

新学算法（10道）

题号	题目	难度	计划日期
LC_0114	二叉树展开为链表	Medium	2026-06-29 周一
LC_0098	验证二叉搜索树	Medium	2026-06-29 周一
LC_0230	二叉搜索树中第 K 小的元素	Medium	2026-06-30 周二
LC_0199	二叉树的右视图	Medium	2026-06-30 周二
LC_0112	路径总和	Easy	2026-07-01 周三
LC_0437	路径总和 III	Medium	2026-07-01 周三
LC_0236	二叉树的最近公共祖先	Medium	2026-07-02 周四
LC_0124	二叉树中的最大路径和	Hard	2026-07-02 周四
LC_0020	有效的括号	Easy	2026-07-03 周五
LC_0155	最小栈	Medium	2026-07-03 周五

复习算法（10道）

题号	题目	轮次	计划日期
LC_0021	合并两个有序链表	R4	2026-06-29 周一
LC_0023	合并 K 个升序链表	R4	2026-06-29 周一
LC_0025	K 个一组翻转链表	R4	2026-06-30 周二
LC_0148	排序链表	R4	2026-06-30 周二
LC_0094	二叉树的中序遍历	R4	2026-07-01 周三
LC_0104	二叉树的最大深度	R4	2026-07-01 周三
LC_0101	对称二叉树	R4	2026-07-02 周四
LC_0226	翻转二叉树	R4	2026-07-02 周四
LC_0102	二叉树的层序遍历	R4	2026-07-03 周五
LC_0105	从前序与中序遍历序列构造二叉树	R4	2026-07-03 周五

进度一览

维度	数值
 周期	2026-06-29 ~ 2026-07-03
 新题总量	10 道
 复习总量	10 道
 每日节奏	2 新 + 2 复
 本周涉及题型	P3 二叉树深入 → P4 栈队列起步

本周围绕题型

- **P3 二叉树 (BST/路径/LCA)** : LC_0114、LC_0098、LC_0230、LC_0199、LC_0112、LC_0437、LC_0236、LC_0124
- **P4 栈队列起步**: LC_0020、LC_0155

 **建议：** 早地铁学习新题，晚地铁复盘，每周末回顾本周所有题目

由 `scripts/generate-detailed-weekly.mjs` 自动生成